

Mini GPU SIMD

Projeto Final CI Digital fase 2

Vinícius Carvalho Pimpim

Campinas, São Paulo

2026

Resumo

Este trabalho apresenta o desenvolvimento e a verificação de uma unidade de processamento vetorial simplificada baseada no paradigma SIMD (*Single Instruction, Multiple Data*). A arquitetura combina 4 unidades aritméticas de 16 bits em uma unidade de processamento central capaz de executar a mesma operação sobre elementos independentes de dois vetores. O sistema inclui memória vetorial de acesso dual-port, interface de requisições por pacotes, decodificação de comandos, escalonamento e controle de execução por máquina de estados. A implementação RTL foi escrita em SystemVerilog e verificada por meio de simulação, testes direcionados a cada módulo e um ambiente UVM com *scoreboard*, cobertura funcional e assertions. São apresentados resultados de síntese, análise temporal e simulações da netlist, incluindo uma execução com anotação SDF.

Palavras-chave: SIMD; GPU; SystemVerilog; UVM; verificação digital; análise temporal.

Abstract

This work presents the development and verification of a simplified vector processing unit based on the SIMD (Single Instruction, Multiple Data) paradigm. The architecture uses four 16-bit arithmetic units to apply the same operation to independent elements of two vectors. The system includes dual-port vector memory, a packet-based request interface, command decoding, scheduling, and finite-state-machine control. The design was described in SystemVerilog and verified using directed tests and a UVM environment with a scoreboard, functional coverage, and assertions. Synthesis, timing analysis, and gate-level simulation results, including SDF annotation, are also presented.

Lista de abreviações e siglas

ALU *Arithmetic Logic Unit* (unidade aritmética e lógica)
AXI *Advanced eXtensible Interface*
FSM *Finite State Machine* (máquina de estados finitos)
GPU *Graphics Processing Unit*
RTL *Register Transfer Level*
SDF *Standard Delay Format*
SIMD *Single Instruction, Multiple Data*
SVA *SystemVerilog Assertions*
UVM *Universal Verification Methodology*

Sumário

1 Introdução **1**
1.1 Contextualização . . . 1
1.2 Objetivos e escopo . . . 1

2 Especificação do Sistema e Protocolo **1**
2.1 Parâmetros e representação dos dados . . . 1
2.2 Conjunto de instruções (ISA) . . . 1
2.3 Formato das requisições . . . 2
2.4 Endereçamento e resposta . . . 2

3 Arquitetura e Implementação RTL **3**
3.1 Visão geral e fluxo de dados . . . 3
3.2 Packet Decoder . . . 3
3.3 Interface de requisições . . . 3
3.4 Memória vetorial dual-port . . . 4
3.5 Scheduler . . . 4
3.6 FSM de execução . . . 4
3.7 Registradores vetoriais . . . 5
3.8 ALU SIMD e unidade aritmética . . . 5

4 Metodologia de Verificação **6**
4.1 Ambiente de simulação e testbench . . . 6
4.2 Ambiente UVM, scoreboard e assertions . . . 6
4.3 Estratégia de testes e cobertura . . . 6

5 Resultados da Verificação **6**
5.1 Evidências e análise . . . 8
5.2 Cobertura e falhas . . . 8

6 Síntese e Análise de Timing **9**
6.1 Fluxo e resultados de síntese . . . 9
6.2 Caminho crítico e simulação Gate-Level . . . 10

7 Limitações, Melhorias Futuras e Conclusão **10**
7.1 Limitações e melhorias futuras . . . 10
7.2 Conclusão . . . 10

1 Introdução

1.1 Contextualização

Aplicações que processam vetores frequentemente realizam a mesma operação sobre vários elementos independentes. Um exemplo claro disso é visto no processo de multiplicação matricial, em que, na etapa de multiplicar uma linha por uma coluna, é realizada a multiplicação elemento a elemento entre dois vetores.

O paradigma SIMD (*Single Instruction, Multiple Data*) explora essa característica ao aplicar uma única instrução a múltiplos dados em paralelo. Essa organização permite estudar como unidades de processamento, memória e controle podem ser integrados para executar operações vetoriais.

Neste trabalho, o conceito SIMD é explorado por meio de uma mini GPU simplificada, composta por quatro *lanes* aritméticas. Cada *lane* representa uma posição no vetor; assim, o núcleo SIMD é capaz de realizar operações simultâneas em um vetor de 4 elementos.

1.2 Objetivos e escopo

O objetivo deste trabalho é projetar, implementar e verificar uma unidade de processamento vetorial capaz de executar as operações de soma, subtração, multiplicação e divisão sobre quatro pares de elementos, juntos compondo dois vetores de 4 posições. A implementação utiliza SystemVerilog e reúne memória vetorial de duas portas, unidades aritméticas paralelas e lógica de controle para coordenar a execução.

A comunicação com o módulo ocorre por meio de requisições de 42 bits enviadas pelo host. Essas requisições são utilizadas para realizar acessos de leitura e escrita na memória, configurar comandos e iniciar operações de processamento. A verificação utiliza testes direcionados a cada módulo e um ambiente UVM, composto por *scoreboard*, cobertura funcional e *assertions*. Nesse projeto também foram realizadas a síntese do circuito RTL e a avaliação da netlist gerada por simulação funcional e temporal em nível *gate-level*.

2 Especificação do Sistema e Protocolo

2.1 Parâmetros e representação dos dados

Para que a MiniGPU realize operações matemáticas, é necessário que ela receba dois vetores de quatro elementos. Cada elemento enviado pelo host possui 8 bits; assim, cada vetor enviado contém 32 bits de dados. Cada elemento do vetor é interpretado como inteiro *signed* em complemento de dois. Na escrita da memória, cada elemento é estendido com sinal para 16 bits. Isso é necessário para comportar resultados que ultrapassem 8 bits, como o produto completo de dois operandos signed de 8 bits. Assim, cada palavra vetorial possui, ao final, 64 bits na memória, divididos em quatro *lanes* de 16 bits.

Tabela 1: Parâmetros principais da Mini GPU SIMD.

Parâmetro	**Especificação**
Número de *lanes*	4
Largura do elemento recebido do host	8 bits, signed
Largura interna de cada *lane*	16 bits, signed
Largura da palavra vetorial	64 bits
Profundidade da memória vetorial	256 palavras
Largura do endereço interno da memória	8 bits
Pacote de requisição	42 bits: `TYPE[41:40]` + `PAYLOAD[39:0]`
Resposta de leitura	64 bits, acompanhada de sinais de validade e erro
Comandos simultâneos	1
Operações	ADD, SUB, MUL e DIV

2.2 Conjunto de instruções (ISA)

As quatro unidades recebem o mesmo *opcode* e operam sobre posições correspondentes dos vetores enviados A e B.

Tabela 2: Conjunto de instruções implementado.

Instrução	Opcode	Operação
ADD	0000 0001	Soma elemento a elemento.
SUB	0000 0010	Subtrai elemento a elemento.
MUL	0000 0011	Multiplica elemento a elemento, preservando o produto de 16 bits para operandos de 8 bits.
DIV	0000 0100	Realiza divisão inteira signed.

Casos de erro e sua sinalização são tratados principalmente pela ALU. O erro é sinalizado quando ocorre divisão por zero ou *overflow* signed. Em caso de divisão por zero, a lane correspondente ao erro retorna zero. Em complemento de dois, uma lane de 16 bits é capaz de representar a faixa de -32768 até 32767. O cenário de divisão de -32768 por -1 causa overflow e produz como resultado o valor `16'h8000`. Os erros das quatro lanes são combinados e registrados para a resposta ao pacote `GPU_START`, comando responsável por sinalizar à GPU o início da operação.

2.3 Formato das requisições

Toda requisição do host utiliza um pacote de 42 bits, dividido em um campo de tipo de 2 bits e cinco campos de 8 bits.

Tabela 3: Formato geral do pacote de requisição.

TYPE	FIELD_0	FIELD_1	FIELD_2	FIELD_3	FIELD_4
[41:40]	[39:32]	[31:24]	[23:16]	[15:8]	[7:0]

Tabela 4: Tipos de requisição.

TYPE	Nome	Função
2'b00	MEM_READ	Solicita a leitura de uma posição da memória vetorial.
2'b01	MEM_WRITE	Escreve um vetor na memória vetorial.
2'b10	GPU_COMMAND	Configura OPCODE, SRC_A, SRC_B e DST.
2'b11	GPU_START	Solicita o início da operação configurada.

Na requisição `GPU_COMMAND`, o campo [39:32] é reservado, seguido por `OPCODE`, `SRC_A`, `SRC_B` e `DST`. Em `MEM_WRITE`, o campo [39:32] indica o endereço de memória em que o vetor será armazenado e os quatro campos `LANE_3` a `LANE_0` carregam o dado do vetor, cada um com 8 bits. Em `MEM_READ`, [39:32] contém o endereço e os 32 bits restantes são reservados. Em `GPU_START`, os 40 bits de payload são reservados.

2.4 Endereçamento e resposta

Como visto, as requisições `MEM_READ` e `MEM_WRITE` fazem uso do `FIELD_0` como índice de 8 bits da memória. Os valores de `8'h00` a `8'hFF` selecionam diretamente as 256 palavras que compõem a memória. Cada posição armazena um valor de 64 bits. Os campos `SRC_A`, `SRC_B` e `DST` presentes na requisição `GPU_COMMAND` utilizam o mesmo espaço de índices para selecionar vetores na memória a serem operados.

A miniGPU envia respostas ao host utilizando três sinais: `rsp_data[63:0]`, que contém o dado da resposta; `rsp_valid`, um pulso que indica quando a resposta está válida; e `rsp_error`, que informa a ocorrência de erro. O host deve observar esses sinais e capturar a resposta quando `rsp_valid` estiver ativo.

O valor esperado do campo `rsp_data` para cada requisição é apresentado abaixo:

- `MEM_READ`: `rsp_data` contém a palavra de 64 bits lida do endereço solicitado.
- `MEM_WRITE`: `rsp_data` retorna zero, confirmando a conclusão da escrita.
- `GPU_COMMAND`: `rsp_data` retorna zero, confirmando o armazenamento dos campos do comando.
- `GPU_START`: após a conclusão da operação, a GPU retorna uma resposta de estado. Nessa resposta, `rsp_data[1]` indica `DONE` e `rsp_data[0]` indica `BUSY`. Como a resposta é enviada após o término, o valor esperado é `DONE = 1` e `BUSY = 0`. O sinal `rsp_error` informa se ocorreu erro aritmético em alguma lane. O vetor resultante fica armazenado no endereço `DST` do comando anterior e pode ser obtido por uma requisição `MEM_READ`.

3 Arquitetura e Implementação RTL

3.1 Visão geral e fluxo de dados

A arquitetura é composta por uma interface de entrada, lógica de decodificação, armazenamento vetorial, controle de execução e quatro unidades aritméticas. A Figura 1 resume os principais blocos e suas conexões funcionais.

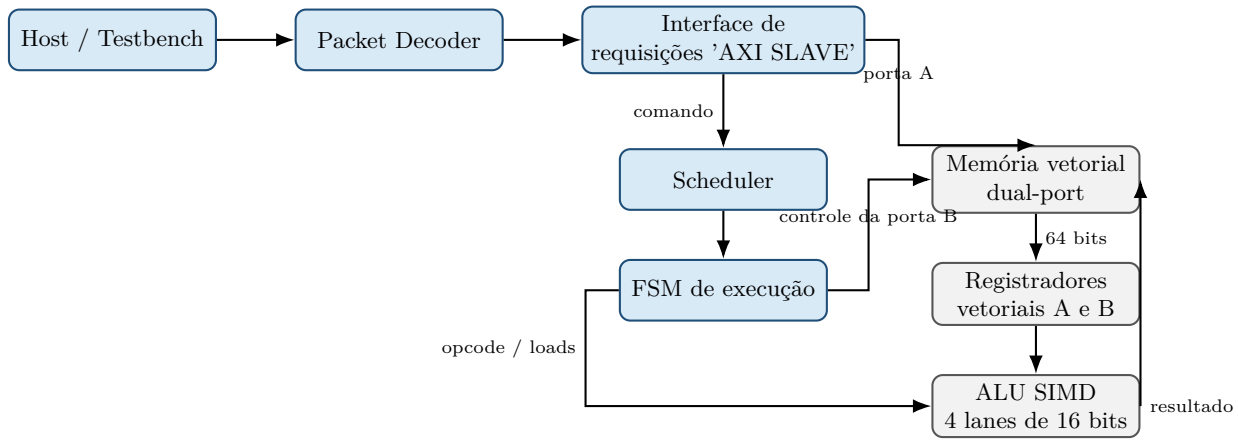


Figura 1: Visão funcional da arquitetura da Mini GPU SIMD.

O módulo `gpu_simd` integra os blocos apresentados na figura. O Packet Decoder recebe a requisição do host e encaminha seus campos ao módulo `axi_slave`. Este módulo interpreta o tipo de pacote e controla a porta A da memória ou encaminha um comando ao Scheduler. A FSM recebe o comando do Scheduler e coordena as leituras dos operandos pela porta B, o carregamento dos registradores, a operação da ALU e a escrita do resultado.

3.2 Packet Decoder

O módulo `packet_decoder` recebe e armazena o pacote de 42 bits do host. Após o handshake de entrada, seus campos permanecem disponíveis até que o módulo seguinte os aceite. Enquanto existe um pacote pendente, o decoder não aceita uma nova requisição.

Tabela 5: Principais sinais do Packet Decoder.

Sinal	Direção	Bits	Função
`clk`, `rst`	Entrada	1 cada	Clock e reset assíncrono ativo em nível alto.
`req_payload`	Entrada	42	Pacote fornecido pelo host.
`req_valid`, `req_ready`	Entrada / saída	1 cada	Handshake de entrada com o host.
`decoded_valid`, `decoded_ready`	Saída / entrada	1 cada	Handshake com o módulo de requisições.
`req_type`, `field_0` a `field_4`	Saída	2 e 5 × 8	Tipo e campos do pacote armazenado.

3.3 Interface de requisições

O módulo `axi_slave` interpreta os quatro tipos de pacote. Em `MEM_WRITE`, forma uma palavra de 64 bits pela extensão de sinal dos quatro campos de dados de 8 bits; em `MEM_READ`, solicita a leitura pela porta A da memória. O pacote `GPU_COMMAND` armazena os campos da operação, enquanto `GPU_START` encaminha o comando ao Scheduler e aguarda sua conclusão antes de responder ao host.

Embora o módulo receba o nome `axi_slave`, sua interface implementada utiliza os sinais de pacote e handshake descritos neste trabalho; ela não expõe os canais de um barramento AXI4-Lite.

Tabela 6: Principais sinais da interface de requisições.

Sinal	Direção	Bits	Função
`clk`, `rst`	Entrada	1 cada	Clock e reset.
`decoded_valid`, `decoded_ready`	Entrada / saída	1 cada	Handshake com o Packet Decoder.
`req_type`, `field_0` a `field_4`	Entrada	2 e 5 × 8	Tipo e campos da requisição.
`rsp_data`, `rsp_valid`, `rsp_error`	Saída	64, 1, 1	Dados, validade e indicação de erro da resposta ao host.
`mem_a_addr`, `mem_a_wdata`, `mem_a_rdata`	Saída / entrada	8, 64, 64	Endereço e dados da porta A da memória.
`mem_a_we`, `mem_a_re`	Saída	1 cada	Habilitações de escrita e leitura da porta A.
`cmd_fields`, `cmd_valid`, `cmd_ready`	Saída / entrada	32, 1, 1	Comando e handshake com o Scheduler.
`cmd_busy`, `cmd_done`, `cmd_error`	Entrada	1 cada	Estado e erro da execução.

3.4 Memória vetorial dual-port

O módulo `dualport_memory` armazena 256 palavras de 64 bits, cada uma contendo quatro elementos de 16 bits. A porta A atende às requisições do host e a porta B é controlada pela FSM durante a execução. As operações de leitura/escrita são síncronas e as saídas de leitura são atualizadas na borda de subida do clock quando a respectiva habilitação está ativa (`a_re`, `b_re`).

A implementação utiliza o padrão comportamental *read-first*, ou seja, uma leitura e uma escrita simultâneas na mesma posição retornam o valor anterior à escrita. Caso as duas portas escrevam simultaneamente no mesmo endereço, a escrita da porta B tem prioridade sobre a da porta A.

Tabela 7: Principais sinais da memória vetorial.

Sinal	Direção	Bits	Função
`clk`	Entrada	1	Clock comum às duas portas.
`a_addr`, `b_addr`	Entrada	8 cada	Endereços das portas A e B.
`a_wdata`, `b_wdata`	Entrada	64 cada	Dados de escrita das portas A e B.
`a_we`, `b_we`	Entrada	1 cada	Habilitações de escrita.
`a_re`, `b_re`	Entrada	1 cada	Habilitações de leitura.
`a_rdata`, `b_rdata`	Saída	64 cada	Dados lidos pelas portas A e B.

3.5 Scheduler

O módulo `scheduler` armazena um comando de 32 bits, composto por opcode e endereços de origem e destino, e o entrega à FSM por um protocolo de handshake `exec_valid/exec_ready`. Seus estados internos são `IDLE`, `DISPATCH`, `EXECUTE` e `DONE`. O sinal `busy` permanece ativo durante os estados `DISPATCH` e `EXECUTE`. O sinal `done` indica a conclusão da operação e permanece ativo até a aceitação do próximo comando.

Tabela 8: Principais sinais do Scheduler.

Sinal	Direção	Bits	Função
`clk`, `rst`	Entrada	1 cada	Clock e reset.
`cmd_fields`, `cmd_valid`, `cmd_ready`	Entrada / saída	32, 1, 1	Comando recebido e handshake de entrada.
`exec_fields`, `exec_valid`, `exec_ready`	Saída / entrada	32, 1, 1	Comando e handshake com a FSM.
`exec_done`	Entrada	1	Indica a conclusão pela FSM.
`busy`, `done`	Saída	1 cada	Estado de execução e conclusão.
`debug_state`	Saída	2	Estado do Scheduler para observação.

3.6 FSM de execução

O módulo `fsm` armazena o comando aceito e gera os sinais de controle para executar a operação. As leituras síncronas dos dois operandos na memória ocorrem em etapas separadas, conforme apresentado na tabela de

estados abaixo. A FSM também controla o carregamento dos registradores, a operação aritmética e a escrita do resultado.

Tabela 9: Sequência de estados da FSM de execução.

Estado	Função
`IDLE`	Aguarda um comando válido do Scheduler.
`READ_A` / `LOAD_A`	Solicita a leitura de `SRC_A` e carrega o registrador A.
`READ_B` / `LOAD_B`	Solicita a leitura de `SRC_B` e carrega o registrador B.
`EXECUTE`	Apresenta o opcode à ALU com os operandos armazenados.
`WRITE_RESULT`	Mantém o opcode e escreve o resultado no endereço `DST`.
`FINISH`	Gera o pulso `exec_done` e retorna a `IDLE`.

Tabela 10: Principais sinais da FSM de execução.

Sinal	Direção	Bits	Função
`clk`, `rst`	Entrada	1 cada	Clock e reset.
`exec_fields`, `exec_valid`, `exec_ready`	Entrada / saída	32, 1, 1	Comando e handshake com o Scheduler.
`mem_b_addr`, `mem_b_we`, `mem_b_re`	Saída	8, 1, 1	Endereço e controle da porta B da memória.
`load_a`, `load_b`	Saída	1 cada	Habilitações dos registradores vetoriais.
`alu_opcode`	Saída	8	Operação aplicada à ALU SIMD.
`exec_done`, `debug_state`	Saída	1, 3	Conclusão e estado da FSM.

3.7 Registradores vetoriais

O módulo `register_vector` contém os registradores dos operandos A e B. Ambos recebem o dado lido pela porta B da memória, mas são carregados separadamente pelos sinais `load_a` e `load_b`, comandados pelo estado da FSM. O reset assíncrono limpa os dois registradores.

Tabela 11: Principais sinais dos registradores vetoriais.

Sinal	Direção	Bits	Função
`clk`, `rst`	Entrada	1 cada	Clock e reset.
`mem_b_rdata`	Entrada	64	Vetor lido da porta B da memória.
`load_a`, `load_b`	Entrada	1 cada	Habilitações de captura dos operandos.
`vector_a`, `vector_b`	Saída	64 cada	Vetores mantidos para a ALU SIMD.

3.8 ALU SIMD e unidade aritmética

O módulo `alu_simd` instancia quatro unidades `alu_unit`. Cada unidade recebe um par de elementos signed de 16 bits e o mesmo código de operação, produzindo um resultado de 16 bits. Os quatro resultados compõem o vetor de saída de 64 bits. A ALU SIMD combina os sinais de erro das lanes por uma operação `OR`; o módulo superior registra esse erro para a resposta de `GPU_START`.

Cada unidade aritmética executa ADD, SUB, MUL ou DIV. Como visto anteriormente, em casos de overflow, o resultado permanece limitado a 16 bits e o erro é sinalizado. Na divisão por zero, a lane produz zero e sinaliza erro.

Tabela 12: Principais sinais da ALU SIMD e de uma unidade aritmética.

Sinal	Direção	Bits	Função
`vector_a`, `vector_b`	Entrada	64 cada	Operandos da ALU SIMD.
`alu_opcode`	Entrada	8	Seleciona a operação aplicada às lanes.
`result`, `error`	Saída	64, 1	Resultado vetorial e erro combinado da ALU SIMD.
`a`, `b`	Entrada	16 cada	Operandos signed de uma unidade aritmética.
`alu_control`	Entrada	4	Operação selecionada para a unidade aritmética.
`result`, `zero`, `error`	Saída	16, 1, 1	Resultado, indicação de zero e erro de uma lane.

4 Metodologia de Verificação

4.1 Ambiente de simulação e testbench

Inicialmente, cada módulo foi testado utilizando um testbench não UVM, ou seja, um arquivo escrito em SystemVerilog para verificar o comportamento funcional isolado dos blocos. Também foram utilizados testbenches para verificar a integração completa da miniGPU. Nesse caso, o host foi modelado pelo testbench, responsável por gerar pacotes de requisição, observar respostas e controlar a sequência das operações.

4.2 Ambiente UVM, scoreboard e assertions

Mais adiante, foi construído um ambiente UVM composto por agente, driver, monitor, ambiente, sequências e testes. O driver aplica os estímulos; o monitor observa as transações; e o scoreboard compara os resultados obtidos com um modelo de referência. Assertions também foram adicionadas e verificam propriedades de handshake, estabilidade dos sinais, controle e sequência de execução.

Tabela 13: Elementos do ambiente de verificação.

Elemento	**Finalidade**
Driver	Receber transações das sequências e transmitir pacotes ao DUT.
Monitor	Observar requisições, respostas e sinais relevantes.
Agent / Environment	Organizar os componentes e a comunicação do ambiente.
Sequences / Tests	Definir cenários direcionados e sequências de execução.
Scoreboard	Comparar resultados do DUT com o modelo de referência.
Assertions (SVA)	Verificar propriedades temporais e protocolos.
Cobertura	Medir os cenários e valores exercitados durante a simulação.

4.3 Estratégia de testes e cobertura

A estratégia de criação do ambiente combina testes de reset, extração de campos, handshake, acesso à memória, configuração e início de comandos, sequência da FSM, operações aritméticas, integração end-to-end, casos extremos, execução back-to-back e estímulos aleatórios. A cobertura funcional mede os tipos de pacote, opcodes, regiões de endereço, sinais de estado e condições de erro definidos no modelo de cobertura. A cobertura de código também foi coletada durante as simulações. Esses casos são apresentados na Tabela 14, que contém o nome e a descrição de cada teste implementado.

5 Resultados da Verificação

A Tabela 14 apresenta os testes UVM criados para o projeto, bem como seus objetivos e as funcionalidades associadas. Nas 17 execuções realizadas, uma para cada teste, os resumos UVM registram zero ocorrências de `UVM_ERROR` e `UVM_FATAL`.

Tabela 14: Lista consolidada de testes e rastreabilidade com as funcionalidades.

Teste	Descrição	Funcionalidades associadas	Prioridade
`test_gpu_reset`	Aplica reset durante o estado ocioso e durante uma operação, verificando o retorno ao estado inicial e a desativação dos sinais de controle.	`GPU_01_RESET`	Alta
`test_packet_field_extraction`	Aplica um pacote com valores conhecidos nos campos e verifica a extração de `TYPE` e de `FIELD_0` a `FIELD_4`.	`PKT_01_HANDSHAKE`, `PKT_02_FIELDS`	Alta
`test_host_backpressure`	Apresenta um segundo pacote `MEM_WRITE` enquanto o Packet Decoder está ocupado e verifica que o host mantém `req_valid` e `req_payload` estáveis até `req_ready` voltar a 1.	`PKT_01_HANDSHAKE`, `PKT_04_UNDERCOMMAND`, `PKT_05_HOST_STABLE`	Alta
`test_packet_mem_write`	Envia um pacote `MEM_WRITE` e verifica `mem_a_addr`, `mem_a_wdata`, `mem_a_we` e a confirmação ao host.	`AXI_02_MEM_WRITE`, `AXI_06_SINGLE_REQUEST`, `AXI_07_RESPONSE`	Alta
`test_packet_mem_read`	Escreve e lê vetores na memória integrada e verifica a ativação de `mem_a_re`; o scoreboard compara as respostas de leitura.	`AXI_03_MEM_READ`, `AXI_07_RESPONSE`	Alta
`test_packet_command_start`	Envia `GPU_COMMAND` seguido de `GPU_START`; verifica os campos entregues ao Scheduler e a ativação de `cmd_valid`.	`AXI_04_COMMAND`, `AXI_05_CONTROL_STATUS`, `AXI_06_SINGLE_REQUEST`, `AXI_07_RESPONSE`	Alta
`test_axi_decode`	Observa a seleção dos caminhos de escrita e leitura de memória e o encaminhamento do comando durante uma operação completa.	`AXI_01_DECODE`, `AXI_04_COMMAND`, `AXI_05_CONTROL_STATUS`	Alta
`test_axi_memory_access`	Escreve e lê vetores pelos endereços inicial, intermediário e final da memória.	`AXI_02_MEM_WRITE`, `AXI_03_MEM_READ`, `MEM_01_PORT_A`	Alta
`test_scheduler_handshake`	Observa os estados `DISPATCH`, `EXECUTE` e `DONE` e os sinais `BUSY` e `DONE`.	`SCH_01_HANDSHAKE` a `SCH_04_SINGLE_COMMAND`	Alta
`test_fsm_sequence`	Observa a sequência dos estados de `READ_A` até `FINISH` durante uma operação.	`FSM_01_ACCEPT` a `FSM_07_DONE`	Alta
`test_gpu_add_end_to_end`	Envia dois pacotes `MEM_WRITE`, um `GPU_COMMAND` com `ADD`, um `GPU_START` e, após o término, um `MEM_READ` para obter o resultado.	`GPU_02_END_TO_END` e integração dos blocos	Alta
`test_gpu_all_ops`	Executa pelo protocolo de pacotes uma operação completa para cada opcode suportado. Os operandos são tratados como valores *signed*; a divisão por zero deve retornar resultado zero, `DONE` e `rsp_error=1`.	`GPU_02_END_TO_END`, `ALU_01_SOMA` a `ALU_04_DIV`, `ALU_06_ERROR`	Alta
`test_gpu_mul_8bit`	Escreve o valor mínimo signed de 8 bits (`8'h80`, ou -128) nas quatro *lanes* de dois vetores, executa `MUL` e verifica o produto `16'h4000` em cada *lane* da resposta de 64 bits.	`GPU_02_END_TO_END`, `AXI_02_MEM_WRITE`, `AXI_03_MEM_READ`, `ALU_03_MULT`, `ALU_05_SIMD`	Alta
`gpu_matrix_mul_test`	Executa a multiplicação de duas matrizes 4x4 por meio de operações SIMD `MUL` e `ADD`, gravando e lendo a matriz resultante pela memória vetorial.	`GPU_02_END_TO_END`, `ALU_01_SOMA`, `ALU_03_MULT`, `ALU_05_SIMD`	Alta
`test_gpu_back_to_back`	Executa operações consecutivas e lê os três destinos para verificar que os resultados anteriores são preservados.	`PKT_04_UNDERCOMMAND`, `AXI_06_SINGLE_REQUEST`, `SCH_03_STATUS`, `GPU_02_END_TO_END`	Média
`test_gpu_address_corner_cases`	Utiliza os endereços 0 e 255, `SRC_A=SRC_B` e `DST` igual a um dos endereços dos operandos.	`MEM_01_PORT_A`, `MEM_02_PORT_B`, `FSM_02_READ_A`, `FSM_03_READ_B`, `FSM_05_WRITEBACK`	Média
`test_gpu_random`	Gera vetores, operações e endereços aleatórios e compara o resultado final com um modelo de referência.	Operações, endereços e dados gerados	Alta

5.1 Evidências e análise

Os logs da suíte de testes registram exemplos de verificação ponta a ponta. Por exemplo, em `test_gpu_mul_8bit`, a multiplicação de −128 por −128 nas quatro lanes produziu `4000_4000_4000_4000`, sem divergências. Na execução de `test_gpu_random`, foram registradas 5.000 transações e 1.000 leituras, sem divergências no scoreboard.

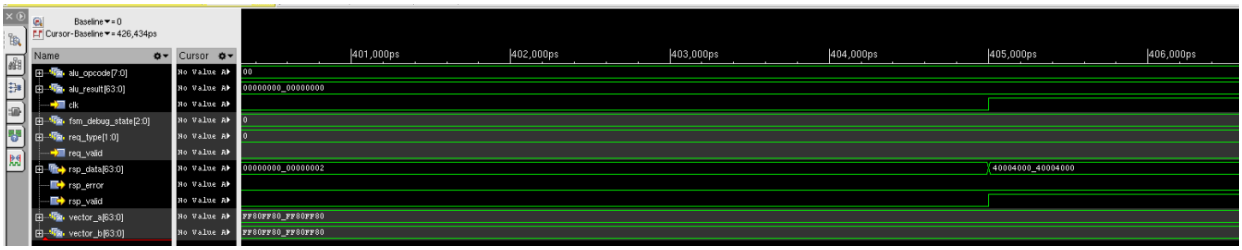


Figura 2: Forma de onda do teste `test_gpu_mul_8bit`. Os vetores de entrada possuem quatro lanes com `FF80` (−128) e a resposta de leitura apresenta `4000_4000_4000_4000`, correspondente ao produto `-128 x -128` nas quatro lanes.

5.2 Cobertura e falhas

A verificação atingiu 100% de cobertura funcional, com 44 dos 44 itens definidos no modelo exercitados. Foram contemplados os tipos de pacote, opcodes, regiões de endereço dos operandos e do destino, o cruzamento entre opcode e região de destino, além dos sinais `DONE` e `rsp_error`. Esse resultado se refere aos cenários definidos no modelo de cobertura funcional.

Na instância principal da GPU, a cobertura de código atingiu 97,72%, com 643 de 658 itens cobertos. Os resultados dos módulos internos foram medidos separadamente.

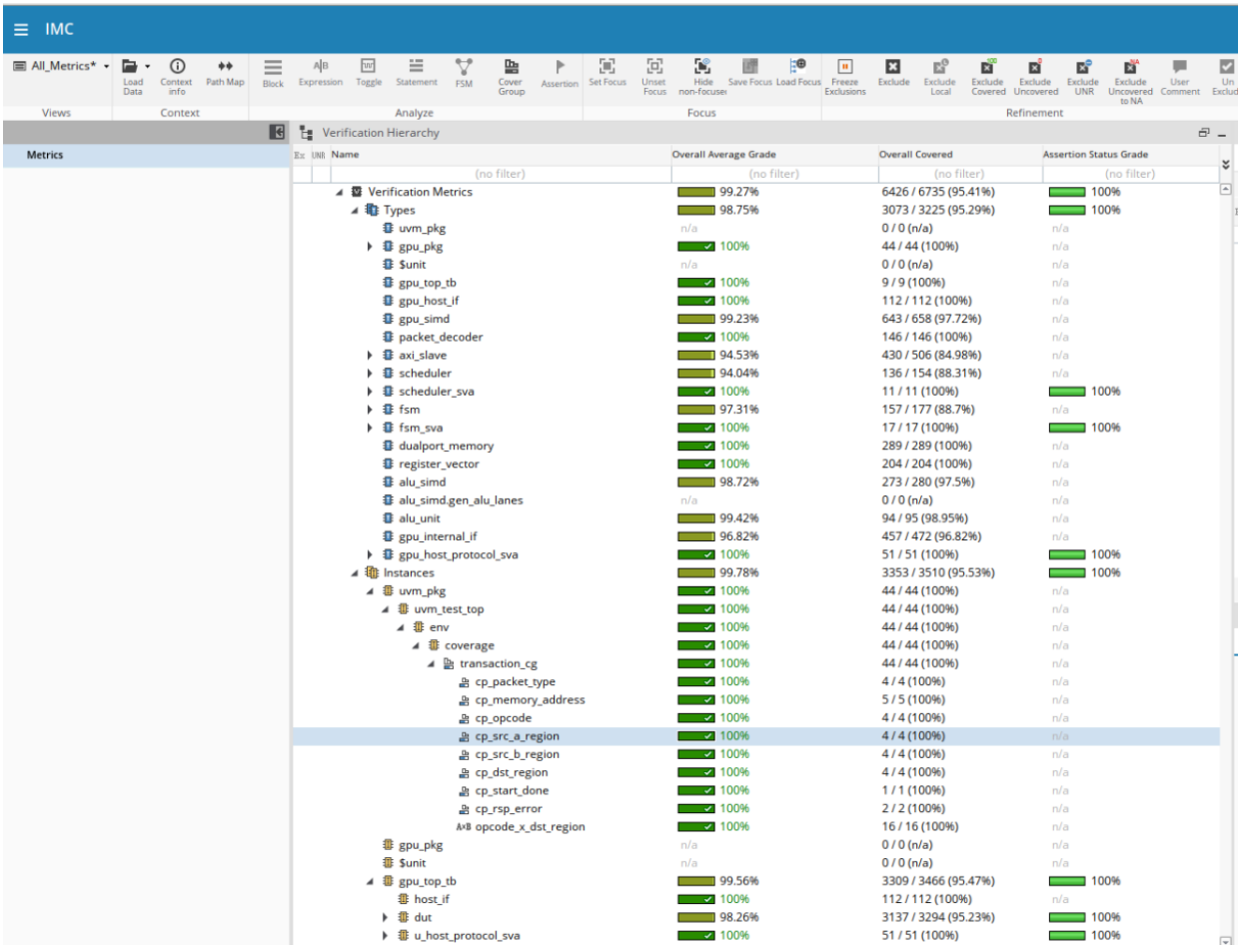


Figura 3: Resumo de cobertura no IMC após a campanha de testes UVM. A instância `gpu_simd` atingiu 643 de 658 itens de código (97,72%) e o modelo de cobertura funcional atingiu 44 de 44 itens (100%).

Durante o desenvolvimento, foi identificada uma falha na inicialização: o driver podia enviar uma requisição antes de o reset estar definido como desativado. O driver foi corrigido para aguardar a liberação do reset antes

de transmitir. Outra melhoria foi a ampliação das lanes e da memória para 16 bits, permitindo preservar o produto completo de dois operandos signed de 8 bits. Na proposta inicial, a memória e as lanes utilizavam 8 bits para armazenar dados e resultados.

Nas execuções dos 17 testes UVM, não foram registrados erros ou falhas fatais.

6 Síntese e Análise de Timing

6.1 Fluxo e resultados de síntese

O fluxo lê o RTL da mini GPU, elabora o projeto e executa síntese genérica e mapeamento para células padrão. Foram utilizadas a biblioteca GPDK045, com tensão de 0,9 V e temperatura de 125 °C, e as restrições temporais do arquivo SDC. Os resultados a seguir são estimativas da síntese.

Tabela 15: Resumo dos resultados de síntese.

Métrica	**Resultado**
Ferramenta / versão	Cadence Genus 22.16-s078_1
Biblioteca / tecnologia	GPDK045, `slow_vdd1v0_basicCells.lib`; canto 0,9 V e 125 °C
Número de células / recursos	81.398 instâncias: 64.565 combinacionais e 16.833 sequenciais
Área estimada	204.149,034 unidades de área da biblioteca para células
Período ou frequência de clock	20 ns / 50 MHz
Restrições relevantes	Incerteza de clock de 1 ns; atrasos de entrada e saída de 1 ns; clock e reset ideais

A memória foi inferida com células padrão, pois o fluxo não utiliza uma macro SRAM compatível. Em um projeto industrial, pode-se utilizar uma macro SRAM com modelos compatíveis para síntese e análise temporal, evitando a implementação do armazenamento com flip-flops. Ao final da síntese, a área reportada para a memória foi de 188.340,084 unidades, cerca de 92,26% da área total de células. A Figura 4 apresenta a imagem exportada a partir dessa netlist no Cadence Genus, permitindo visualizar a organização dos blocos e suas conexões.

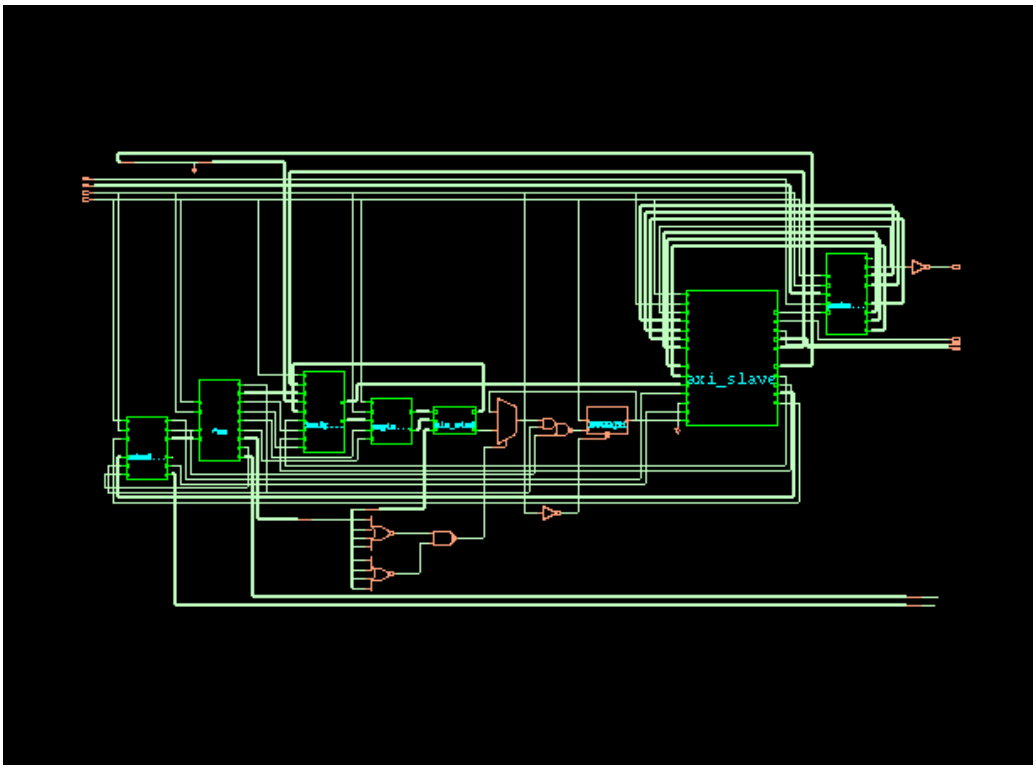


Figura 4: Esquemático exportado da netlist sintetizada no Cadence Genus.

6.2 Caminho crítico e simulação Gate-Level

O relatório de síntese identifica um caminho de setup entre um registrador do operando B e um registrador da memória vetorial. O caminho atravessa lógica de divisão de uma lane da ALU e apresenta slack positivo sob a restrição de 20 ns. Esses valores correspondem à análise da netlist sintetizada nas condições e restrições adotadas.

Tabela 16: Resumo da análise temporal.

Item	Resultado
Ferramenta / relatório	Cadence Genus 22.16-s078_1; `gpu_simd_timing.rpt`
Caminho crítico (origem / destino)	`vector_b_reg[48]/CK` / `mem_reg[254][63]/D`
Atraso do caminho de dados	16,055 ns
Slack / resultado do setup	2,836 ns; restrição atendida
Lógica no caminho	Lógica de divisão da lane 3 e seleção para escrita na memória
Simulação Gate-Level	Funcional sem SDF: teste registrado como aprovado; simulação com SDF: teste registrado como aprovado
Anotação SDF	`gpu_simd.sdf`, valores máximos; 239.898 de 239.914 atrasos de caminho anotados

A análise temporal também foi avaliada por simulação da netlist com anotação dos atrasos do arquivo SDF. Conforme o registro do fluxo, o Cadence Genus gerou o SDF e o Xcelium anotou os valores máximos de atraso na instância da GPU. O testbench executou com período de clock de 20 ns, permitindo observar o comportamento funcional da implementação mapeada sob os atrasos anotados. Não houve violações temporais registradas.

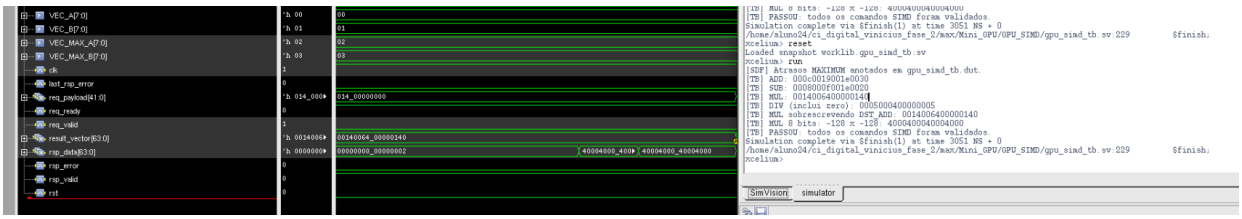


Figura 5: Simulação temporal da netlist no SimVision. O console confirma a anotação dos atrasos SDF `MAXIMUM` em `gpu_simd_tb.dut`; a forma de onda mostra a resposta `4000_4000_4000_4000` e a execução termina aprovada em 3051 ns.

7 Limitações, Melhorias Futuras e Conclusão

7.1 Limitações e melhorias futuras

A implementação opera com quatro lanes, um único comando de cada vez e quatro operações aritméticas. A interface do host permite escrever operandos de 8 bits, enquanto os resultados internos utilizam lanes de 16 bits; operações sucessivas podem produzir overflow. O modelo de memória foi sintetizado em células padrão, com impacto expressivo na área estimada.

Como trabalhos futuros, podem ser estudados o uso de uma SRAM compatível, a otimização do caminho de divisão e a ampliação do conjunto de operações ou da capacidade de processamento. Neste projeto inicial, apenas uma instrução é processada por vez. Um estudo futuro pode expandir esse cenário, utilizando filas no Scheduler para guardar as requisições na sequência em que chegam.

7.2 Conclusão

O projeto integrou memória vetorial, controle e quatro unidades aritméticas em uma mini GPU SIMD descrita em SystemVerilog. Os logs capturados do ambiente registraram a execução dos testes UVM sem erros ou falhas fatais, incluindo operações completas e multiplicação de matrizes. O modelo de cobertura funcional atingiu 44 de 44 itens definidos.

A síntese sob a restrição de 20 ns apresentou slack positivo de aproximadamente 2,836 ns no caminho crítico reportado. A simulação temporal também registrou conclusão funcional com anotação SDF e sem violações temporais. Esses resultados demonstram o comportamento observado nas condições avaliadas e oferecem uma base para aperfeiçoar a arquitetura e aprofundar sua implementação física.